

Student Name:	Brandon Sandhu	Course:	Mathematics (MMath) G103
University Number:	2124562	Year of Study:	4

Contents

1	Introduction	1
2	First Steps	1
2.1	Serial Optimisations	7
3	Parallel Implementation	10
3.1	OpenMP Implementation	12
3.2	OpenMP Scalability Issue	13
3.3	Comparing OpenMP Strategies	13
3.4	OpenMP Performance Measurements & Analysis	13
3.4.1	Statistics for 500 Stars on 100000 steps	14
3.5	MPI & OpenMP Hybrid Implementation	16
3.6	MPI & OpenMP Hybrid Performance	19
3.6.1	Statistics for 500 Stars on 100000 steps	19
3.6.2	Statistics for 600 Stars on 100000 steps	20
3.6.3	Statistics for 750 Stars on 100000 steps	21
3.6.4	Plots of speed up, Karp-Flatt Metric and efficiency against processors	22
3.6.5	Analysis of plots	24
3.7	Full Scale Simulation	24
4	Conclusion	25

1 Introduction

The assignment presents code which implements a gravitational ($N + 1$)-body simulation modelling stars in a global cluster, in the vicinity of a very large black hole. Our goal is to complete the serial code by correctly implementing missing loops involving the various steps of the velocity-Verlet (VV) algorithm, described below:

$$v_i^k \left(t + \frac{1}{2} \Delta t \right) = v_i^k(t) + \frac{1}{2} a_i^k(t) \Delta t,$$

(1)

$$r_i^k(t + \Delta t) = r_i^k(t) + v_i^k \left(t + \frac{1}{2} \Delta t \right) \Delta t,$$

(2)

$$v_i^k(t + \Delta t) = v_i^k \left(t + \frac{1}{2} \Delta t \right) + \frac{1}{2} a_i^k(t + \Delta t) \Delta t,$$

(3)

which are used to evolve the position and velcoity of stars; add parallelism using a hybrid of OpenMP and MPI and then measure the performance of our strategy whilst reporting on the results and the various methods we have used and explaining why.

2 First Steps

We are tasked to evolve the star over time using just the acceleration of the Black Hole’s gravity. This includes adding appropriate loops to generate the action of equations (1) - (3), on the `pos`, `vel` and `acc` arrays. The given code seems to already implement these loops by firstly advancing the velocities `vel` through the first half step in (1) using the loop:

Listing 1: For loop for computing (1) of the velocity-Verlet algorithm

```

for (istar=0;istar<nStars;istar++) {
    double *myvel = &vel[istar*3];
    double *myacc = &acc[istar*3];

    /* First half step in velocities */
    myvel[x] += 0.5*delta_t*myacc[x];
    myvel[y] += 0.5*delta_t*myacc[y];
    myvel[z] += 0.5*delta_t*myacc[z];
}

```

It then proceeds to advance the positions, `pos`, through a full step in (2) using the loop:

Listing 2: For loop for computing (2) of the velocity-Verlet algorithm

```

for (istar=0;istar<nStars;istar++) {
    /* Don't move stars which are inactive */
    if (active[istar]==0) continue;

    double *mypos = &pos[istar*3];
    double *myvel = &vel[istar*3];

    /* Full step in positions */
    mypos[x] += delta_t*myvel[x];
    mypos[y] += delta_t*myvel[y];
    mypos[z] += delta_t*myvel[z];
}

```

Then, the third step in (3) requires advancing the acceleration, `acc` through a full step. For this the following equation is used:

$$a_i^k = -r_i^k \cdot \frac{GM_E}{|\mathbf{r}_i|^3} + \sum_{j \neq i} (r_j^k - r_i^k) \cdot \frac{Gm_j}{|\mathbf{r}_{ij}|^3} \quad (4)$$

where $\mathbf{r}_{ij}$ represents the vector separating star i and star j . This is implemented in the serial code in two steps. Firstly the first term is already dealt with in a for loop described below:

Listing 3: For loop for computing the first term of acceleration into the array `acc`

```

for (istar=0;istar<nStars;istar++) {
    double *mypos = &pos[istar*3];
    double *myvel = &vel[istar*3];
    double *myacc = &acc[istar*3];

    /* Vector separating current star from origin */
    double rvec[3];

    rvec[x] = mypos[x];
    rvec[y] = mypos[y];
    rvec[z] = mypos[z];

    double radsq = rvec[x]*rvec[x] + rvec[y]*rvec[y] + rvec[z]*rvec[z];
    double rad = sqrt(radsq);

    /* Unit vector */
    double rhat[3];
    rhat[x] = rvec[x]/rad;
    rhat[y] = rvec[y]/rad;
    rhat[z] = rvec[z]/rad;
}

```

```

double Amag = BigG*BlackHoleMass/radsq;

myacc[x] = -Amag*rhat[x];
myacc[y] = -Amag*rhat[y];
myacc[z] = -Amag*rhat[z];
}

```

and then we are later tasked to implement the computations for the second term of (4) in a double for loop. Additionally, the code also considers changes in acceleration due to a *drag* term which is described below:

$$F_{i,\text{drag}}^k = -\frac{1}{2}\rho_{\text{dust}}v_i^k \cdot |\mathbf{v}_i| \cdot A_i \quad (5)$$

where ρ_{dust} is the density of the dust, A_i is the cross-sectional area of the star, and $\mathbf{v}_i$ is the velocity of the star. This is implemented in the following for loop:

Listing 4: For loop for computing the effect of drag on acceleration

```

for (istar=0;istar<nStars;istar++) {
    double *mypos = &pos[istar*3];
    double *myvel = &vel[istar*3];
    double *myacc = &acc[istar*3];

    /* Compute height above black hole event horizon */
    double radsq = mypos[x]*mypos[x] + mypos[y]*mypos[y] + mypos[z]*mypos[z];
    double distance = sqrt(radsq);

    /* Compute speed */
    double velsq = myvel[x]*myvel[x] + myvel[y]*myvel[y] + myvel[z]*myvel[z];
    double speed = sqrt(velsq);

    /* Get drag force - function in data.c */
    double fDrag = dragForce(radius[istar],speed,distance,BHRadius);

    /* Unit vector in direction of velocity */
    double vhat[3];
    vhat[x] = myvel[x]/speed;
    vhat[y] = myvel[y]/speed;
    vhat[z] = myvel[z]/speed;

    /* Add in acceleration due to drag */
    myacc[x] += -fDrag*vhat[x];
    myacc[y] += -fDrag*vhat[y];
    myacc[z] += -fDrag*vhat[z];
}

```

After the complete computation of acceleration, we can compute step (3) of the velocity-Verlet algorithm which is already implemented as a for loop described below:

Listing 5: For loop for computing (3) of the velocity-Verlet algorithm

```

for (istar=0;istar<nStars;istar++) {
    double *myvel = &vel[istar*3];
    double *myacc = &acc[istar*3];

    /* First half step in velocities */
    myvel[x] += 0.5*delta_t*myacc[x];
    myvel[y] += 0.5*delta_t*myacc[y];
    myvel[z] += 0.5*delta_t*myacc[z];
}

```

We are tasked to add print statements which output the x , y and z coordinates of the star over every 10 steps so we can plot the orbit of the singular star in the code so far. We achieve this by adding

```
if (step % 10 == 0) {  
    printf("Step %d, Star %d: x = %13.6e, y = %13.6e, z = %13.6e\n", step, istar, mypos[x], mypos[y],  
        mypos[z]);  
}
```

at the end of the for loop concerning the drag. Since that loop is nested within an outer loop which iterates the steps, the logic reads that for every star, if the current step is a multiple of 10 then output the position of that star which is precisely what we require. We add a plot showing this orbit in the $x - y$ plane:

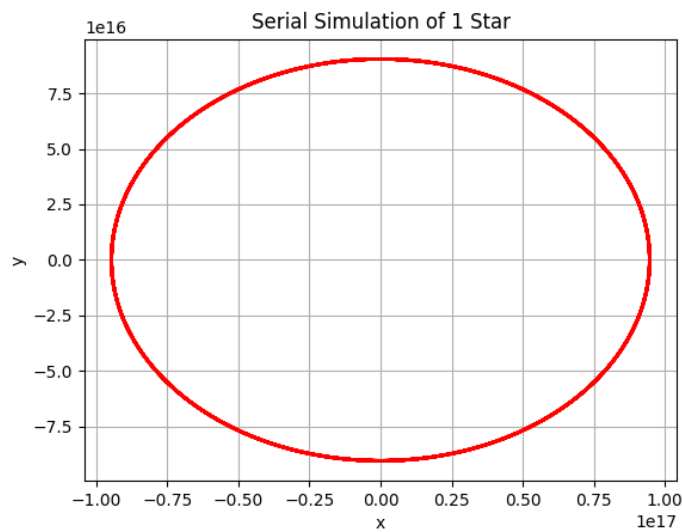


Figure 1: Orbit of a singular star using serial code implementation in the $x - y$ plane.

We now set `nStars` to 5, and complete the double for loop which adds the second term of acceleration in (4). This is achieved similar to the first term, with the following code:

Listing 6: For loop for computing the second term of acceleration into the array `acc`

```
for (istar = 0; istar < nStars; istar++) {
    double *ipos = &pos[istar*3];
    double *iacc = &acc[istar*3];

    for (jstar = 0; jstar < nStars; jstar++) {
        // Skip self-interactions since i != j
        if (istar == jstar) continue;

        double *jpos = &pos[jstar*3];
        double *jacc = &acc[jstar*3];

        // Vector separation between stars i and j
        double rvec[3];
        rvec[x] = jpos[x] - ipos[x];
        rvec[y] = jpos[y] - ipos[y];
        rvec[z] = jpos[z] - ipos[z];

        double dotProduct = rvec[x]*rvec[x] + rvec[y]*rvec[y] + rvec[z]*rvec[z];
        double rvecSize = sqrt(dotProduct);

        // Gravitational acceleration due to star j on star i
        double Amj = BigG * mass[jstar] / dotProduct;

        iacc[x] += Amj * rvec[x] / rvecSize;
        iacc[y] += Amj * rvec[y] / rvecSize;
        iacc[z] += Amj * rvec[z] / rvecSize;
    }
}
```

The specification also states that we should implement the velocity Verlet algorithm such that stars flagged as inactive should not have their positions updated. This is already in the loop that computes positions as shown in Listing (2). Specifically, the lines

```
/* Don't move stars which are inactive */
if (active[istar]==0) continue;
```

ensure we skip any iteration on any inactive star. However, their mass and position should still be used when computing the gravitational acceleration of all other stars so the loop that controls acceleration computation should not have any such restrictions.

Additionally, since inactive stars now have a fixed position, their velocities should remain fixed too, therefore we must add a similar `continue` clause to the for loop in Listings (1) and (5) concerning steps (1) and (3) of the velocity-Verlet algorithm. This is achieved with the following line of code:

```
/* Don't change velocity of stars which are inactive */
if (active[istar]==0) continue;
```

which is added to those two for loops. Since inactive stars have their position and velocities fixed, there should be no more changes to their acceleration so computing accelerations of such inactive stars is redundant. Therefore, we can add a similar `continue` clause to acceleration computations to save computational resources and improve efficiency. Specifically, we add

```
/* Don't update acceleration of stars which are inactive */
if (active[istar]==0) continue;
```

to the for loops in Listings (3), (6) and (4) which compute the first, second term of acceleration in (4) and the acceleration change due to drag in (5), respectively.

After implementing these changes, we now add a timer around the outer most timesteps loop by starting the timer with `clock_t start = clock()` and ending the timer with `clock_t end = clock()` and then computing the elapsed time using `double elapsed_time = (double)(end - start) / CLOCKS_PER_SEC`. This allows us to get a baseline time for the unoptimised, serial code with `nStars = 5` on `vonneumann` (with `Nsteps = 1000000` fixed).

The baseline time is 1.483582 seconds averaged over 5 runs.

The two main parameters that control the computation times in our algorithm are `Nsteps`, which controls the outer most loop in an expected linear complexity, and `nStars`, which has expected complexity of $\mathcal{O}(n^2)$ due to the double for loop when computing acceleration due to interactions between stars. Let us time the unoptimised serial code on `avon` for various choices of these parameters.

Change	100000 Steps	1000000 Steps	2000000 Steps	4000000 Steps	5000000 Steps
5 Stars	0.096547	0.957087	1.937471	3.889281	4.786223
10 Stars	0.288011	2.879677	5.759455	11.519103	14.392431
15 Stars	0.595647	5.852486	11.660181	23.633229	31.156308

Table 1: Timing unoptimised serial code with various parameters

The specification asks us to accurately time various segments of the code to determine where time is spent. For this, we implement timers around each of the for loops for various parts of computations that occur within a single step, and then sum the contributions from each iteration up to construct a distribution of where time was spent during the simulation. These results are displayed for various values of `nStars` below:

5 Stars	10 Stars	50 Stars
===== Performance Report =====	===== Performance Report =====	===== Performance Report =====
Total Simulation Time: 4.477347 seconds	Total Simulation Time: 6.471822 seconds	Total Simulation Time: 56.881463 seconds
Position Updates: 0.289006 s (6.45%)	Position Updates: 0.336508 s (5.20%)	Position Updates: 0.662876 s (1.17%)
Velocity (Half 1): 0.295914 s (6.61%)	Velocity (Half 1): 0.347498 s (5.37%)	Velocity (Half 1): 0.688867 s (1.21%)
Velocity (Half 2): 0.295522 s (6.60%)	Velocity (Half 2): 0.348889 s (5.39%)	Velocity (Half 2): 0.689225 s (1.21%)
BH Acceleration: 0.433274 s (9.68%)	BH Acceleration: 0.578778 s (8.94%)	BH Acceleration: 1.638596 s (2.88%)
Star Interactions: 0.665180 s (14.86%)	Star Interactions: 2.051113 s (31.69%)	Star Interactions: 48.259531 s (84.84%)
Drag: 0.475976 s (10.63%)	Drag: 0.704517 s (10.89%)	Drag: 2.377531 s (4.18%)
Bookkeeping: 7.219052 s (7.22%)	Bookkeeping: 5.996889 s (6.00%)	Bookkeeping: 1.454393 s (1.45%)
Misc: 1.699253 s (37.95%)	Misc: 1.716411 s (26.52%)	Misc: 1.737557 s (3.05%)
=====	=====	=====

Figure 2: Performance Reports for Different Star Counts Before Serial Optimisation

From the above figure, we observe that for larger values of `nStars`, the most computationally expensive section of the code is the double for loop involving interactions between stars. One also observes that the computational complexity of this double for loop grows non-linearly as we increase `nStars`. Since the loop concerning the interaction between stars has complexity $\mathcal{O}(n^2)$ with $n = \text{nStars}$, we expect a quadratic increase in time as `nStars` increases. This is shown clearly in the following figures:

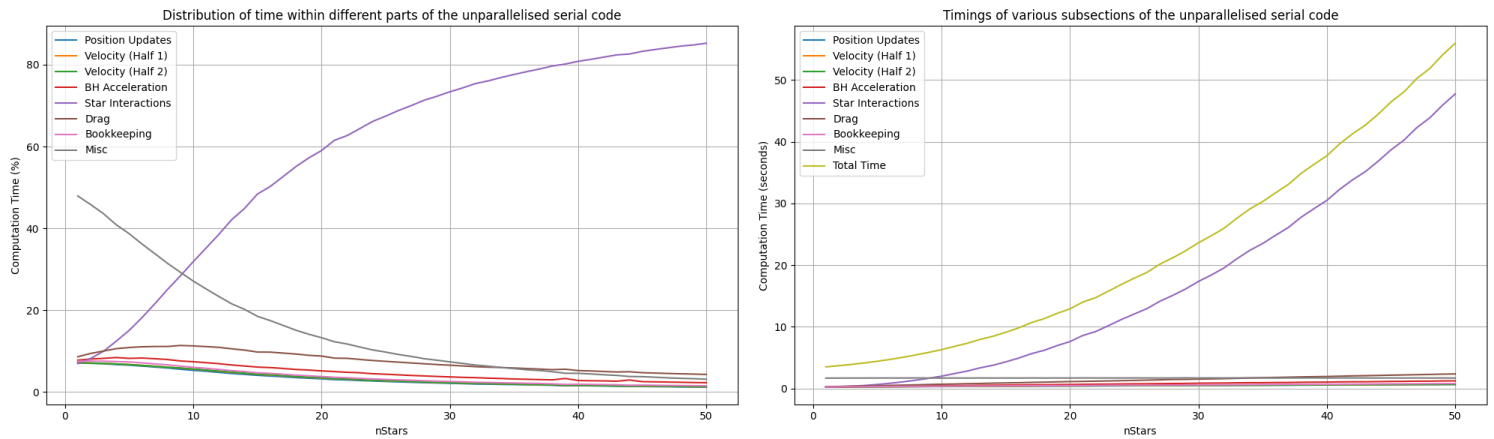


Figure 3: Distribution of time within different parts of the unparallelised serial code.

2.1 Serial Optimisations

We next add some serial code optimisations and document the changes below and measure the performance on *vonneumann*.

Change	Explanation	Run Time (5 Stars)	Run Time (10 Stars)	Run Time (15 Stars)
No Change: Original serial code with no optimisations	-	0.905553	2.803205	5.619873
Factored Terms: factored <code>0.5*delta_t</code> out of timesteps loop	The term <code>0.5*delta_t</code> is used in the velocity loops, namely Listings (1) and (5). We factor it to reduce floating point operations.	0.899865	2.763186	5.587237
Reduced divisions: Replaced multiple divisions by multiplication with reciprocal	The for loops in Listings (3), (4) and the intensive double loop (6) all contain three divisions which I replaced with multiplications by a precomputed reciprocal.	0.900311	2.776508	5.91372
Reduced pointer dereferencing: In the double for loop, we reduce the calls made to pointers from main memory	We introduce temporary variables to sum the contributions of acceleration between interacting stars and then add the temporary variables to the pointers once in the outer loop.	0.906293	2.710157	5.435092
Intermediate variables: In the double for loop, we reduce the calls made to arrays and use variables	Reading from arrays can take considerably longer because of memory access patterns so using variables where possible is an important serial optimisation. We apply this to Listings (3) and (6), the acceleration computation loops.	0.812766	2.322879	4.571229
Intermediate variables: In the dust computation loop, we read <code>mypos</code> and <code>myvel</code> to temporary variables.	These variables are used multiple times so we read them to temporary variables to reduce the number of calls to the pointer.	0.806401	2.292738	4.536462
Direct array access: Remove introducing variables <code>x</code> , <code>y</code> , <code>z</code> for referencing.	These variables cause overhead in retrieving data from main memory so by explicitly using 0, 1 and 2 in place of <code>x</code> , <code>y</code> and <code>z</code> optimises our performance.	0.738892	2.121060	4.163233
Algorithm replacement for sqrt: Suitable approximation algorithm.	The <code>sqrt</code> function is expensive and occurs in the double for loop. We could use an approximation such as a Taylor series approximation.	-	-	-
Remove branching within double for loop Split into two separate loops to avoid branching in one loop	We split the inner loop (excluding <code>jstar = istar</code>) into two to eliminate branching, enabling better compiler optimizations. Additionally, we cache <code>ipos[0]</code> , <code>ipos[1]</code> , and <code>ipos[2]</code> in temporary variables for reuse.	0.670385	1.849180	3.565062
Increased optimisation flag -O3	We apply the compiler flag <code>-O3</code> to enforce more compiler level optimisations on the serial code.	0.241420	0.666586	1.292138

Table 2: Serial optimisations made to the code and timings

We measure the optimised serial code on `avon` using various choices for `Nsteps` and `nStars` as shown in the table below:

Next, we compare the speed up of the code as a result of our serial optimisations thus far. This is shown by

Change	100000 Steps	1000000 Steps	2000000 Steps	4000000 Steps	5000000 Steps
5 Stars	0.071881	0.746225	1.459178	2.972118	3.593652
10 Stars	0.206466	2.028419	4.106372	8.106538	10.185462
15 Stars	0.396359	3.940103	7.939402	15.827934	19.843810

Table 3: Timing optimised serial code with various parameters (without -O3 flag)

comparing tables (1) and (3) through the following plots: One observes immediately that the computation time grows

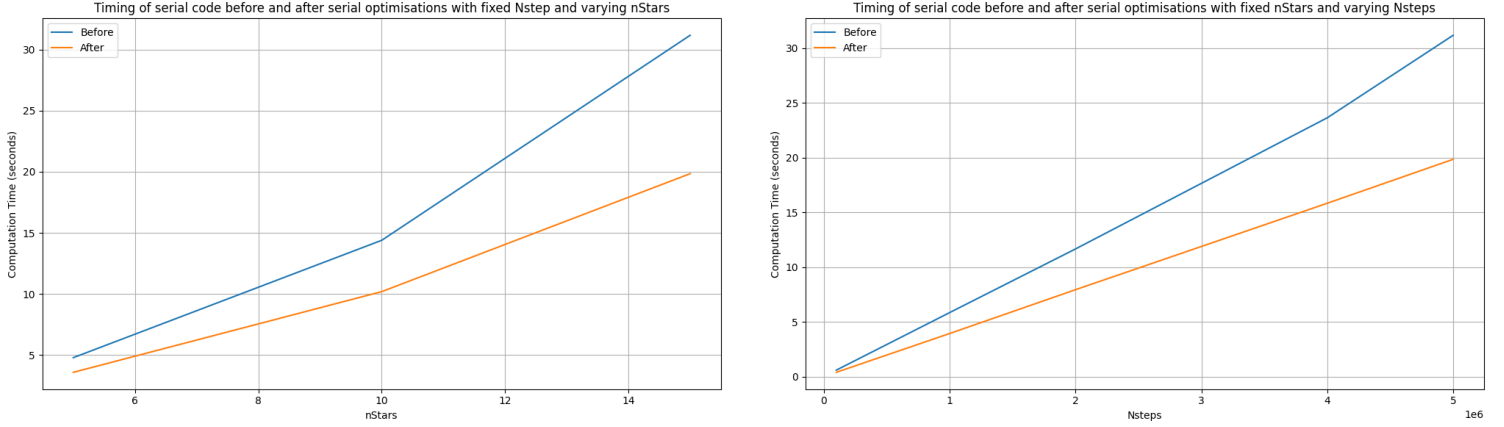


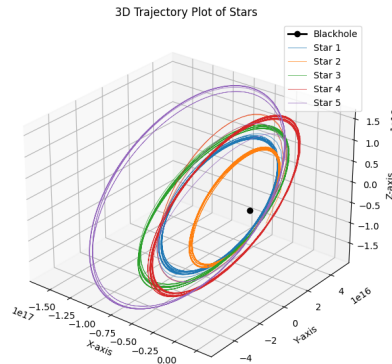
Figure 4: Left: Timings of serial code before (blue) and after (orange) serial optimisations with fixed `Nstep` = 5000000 and varying `nStars`. Right: Timings of serial code before (blue) and after (orange) serial optimisations with fixed `nStars` = 15 and varying `Nsteps`.

quadratically as we vary `nStars`, as expected due to the double for loop when computing interactions between stars. One also observes a linear speed up when increasing `Nsteps`. We also observe a significant performance increase after applying our serial optimisations. Since we have established the relationships between `nStars`, `Nsteps` and computation time, we will give timings with the -O3 flag for fixed `Nsteps` = 5000000 and `nStars` = 5, 10, 15 below for completeness.

5 Stars	10 Stars	15 Stars
1.452766	4.249769	8.384495

Table 4: Timing optimised serial code with fixed `Nsteps` = 5000000 and `nStars` varying, with -O3 flag.

For testing reproducibility and correctness of the code in the future, we run the simulation with the following parameters: `./gravity -N 5 -M 100 -A 10 -P 2 -R 0.2 -S 1000000`. This produces the following output:

Figure 5: Trajectory plot of 5 stars with parameters `-M 100 -A 10 -P 2 -R 0.2 -S 1000000`.

We now remove all print statements initially provided because we draw our attention to parallelising the code.

3 Parallel Implementation

We import OpenMP using `#include <omp.h>` and change the code for starting our clock based on whether OpenMP is in use or not. This is achieved with the following code:

```
#ifndef _OPENMP
    /* Use clock_gettime */
    clock_gettime(CLOCK_PROCESS_CPUTIME_ID,&ti);
    start = (double)ti.tv_sec + ti.tv_nsec/1.0E9;
#elseif _OPENMP
    /* Use OpenMP clock */
    start = omp_get_wtime();
#endif
```

and

```
#ifndef _OPENMP
    /* Use clock_gettime */
    clock_gettime(CLOCK_PROCESS_CPUTIME_ID,&tf);
    end = (double)tf.tv_sec + tf.tv_nsec/1.0E9;
#elseif _OPENMP
    /* Use OpenMP clock */
    end = omp_get_wtime();
#endif
```

Next, we combine some loops together to reduce the total number of loops we have to parallelise, this means we can use less parallel regions and therefore reduce parallel overhead which will improve performance. Firstly, we combine the for loops for computing steps (1) - (2) of the velocity-Verlet algorithm. This looks like the following:

Listing 7: Combined for loop for computing steps (1) - (2) of the velocity-Verlet algorithm

```
for (istar=0;istar<nStars;istar++) {
    /* Don't change velocity of stars which are inactive */
    if (active[istar]==0) continue;

    double *myvel = &vel[istar*3];
    double *myacc = &acc[istar*3];
    double *mypos = &pos[istar*3];

    /* First half step in velocities */
    myvel[0] += dt_half*myacc[0];
    myvel[1] += dt_half*myacc[1];
    myvel[2] += dt_half*myacc[2];

    /* Full step in positions */
    mypos[0] += delta_t*myvel[0];
    mypos[1] += delta_t*myvel[1];
    mypos[2] += delta_t*myvel[2];
}
```

This also provides a serial optimisation since we have less branching and less iterations. We cannot combine the double for loop concerning interactions between stars into this combined for loop, since they contain dependencies. However, the we can group the loops concerning acceleration computations namely Listings (3), (4), (6) into a single for loop. Additionally, the loop computing (3) of the velocity-Verlet algorithm given in Listing (5) can be combined into this

loop too since it only depends on the acceleration at any given point. Therefore, our second main loop is given by:

Listing 8: Combined for loop for computing acceleration changes to the stars and the second step of the velocity-Verlet algorithm specifically (3)

```

for (istar = 0; istar < nStars; istar++) {
    if (active[istar] == 0) continue;
    double *myacc = &acc[istar * 3];
    double *mypos = &pos[istar * 3];
    double *myvel = &vel[istar * 3]; // Used first for drag acceleration

    double ix = mypos[0];
    double iy = mypos[1];
    double iz = mypos[2];

    /* Vector separating current star from origin */
    double radsq = ix * ix + iy * iy + iz * iz;
    double Amag = (BigG * BlackHoleMass) / (sqrt(radsq) * radsq);

    myacc[0] = -Amag * ix;
    myacc[1] = -Amag * iy;
    myacc[2] = -Amag * iz;

    /*-----*/
    /* Compute accelerations due to interactions */
    /* between stars. Requires double loop.      */
    /*-----*/

    double xsum = 0.0, ysum = 0.0, zsum = 0.0;

    for (int jstar = 0; jstar < nStars; jstar++) {
        if (jstar == istar) continue; // Skip self-interaction

        double *jpos = &pos[jstar * 3];

        // Separation vector r_ij
        double rx = jpos[0] - ix;
        double ry = jpos[1] - iy;
        double rz = jpos[2] - iz;

        // Calculating norm^3 of r_ij
        double rvec_radsq = rx * rx + ry * ry + rz * rz;
        double rvec_rad3_inv = mass[jstar] / (sqrt(rvec_radsq) * rvec_radsq);

        // Incrementing sums
        xsum += rx * rvec_rad3_inv;
        ysum += ry * rvec_rad3_inv;
        zsum += rz * rvec_rad3_inv;
    }

    // Updating acceleration with sums
    myacc[0] += BigG * xsum;
    myacc[1] += BigG * ysum;
    myacc[2] += BigG * zsum;

    /*-----*/
    /* Compute drag due to dust      */
    /*-----*/
    double velx = myvel[0];
    double vely = myvel[1];

```

```

double velz = myvel[2];

/* Compute height above black hole event horizon */
double distance = sqrt(radsq);

/* Compute speed */
double velsq = velx * velx + vely * vely + velz * velz;
double speed = sqrt(velsq);

/* Get drag force - function in data.c */
double fDrag = dragForce(radius[istar], speed, distance, BHRradius);

// Scale factor for acceleration update
double scale = -fDrag / speed;

/* Add in acceleration due to drag */
myacc[0] += scale * velx;
myacc[1] += scale * vely;
myacc[2] += scale * velz;

/*-----*/
/* Advance velocities through second half step */
/*-----*/
myvel[0] += dt_half*myacc[0];
myvel[1] += dt_half*myacc[1];
myvel[2] += dt_half*myacc[2];
}

```

Finally, we have the book keeping loop which only requires finding the minimal position, by computing it as a radius, and since positional data is completely updated in our first loop in Figure (7), we can also include the book keeping logic into the loop of Listing (8).

Therefore, the structure of our serial code has simplified considerably. We have a main outer loop which iterates the timesteps and within this we have two subloops. The first subloop iterates over the first step of the velocity-Verlet algorithm given by Listing (7). The second subloop iterates over the rest of the computations including acceleration calculations, the second step in the velocity-Verlet algorithm and any book keeping, given by Listing (8).

3.1 OpenMP Implementation

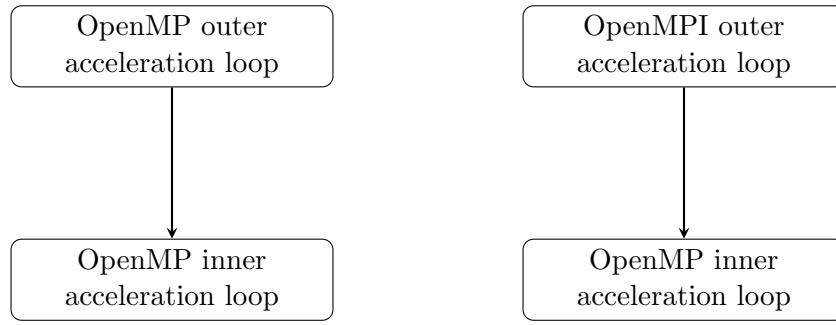
Firstly, we observe that the outer most loop has a large number of iterations so by parallelising any subloop will cause a large amount of overhead. Consequently, we must be careful to not cause too much parallel overhead that it slows our code down more than the benefits of applying parallelism to the loops. For this, we must identify the most computationally expensive loop and try parallelising that. We combined loops before so that we only need to invoke parallel regions where necessary instead of multiple times which would introduce unnecessary overhead. As seen in (12), the loop containing acceleration computations is the most expensive and thus we will focus on parallelising this loop. This would require finding an optimal balance between the number of threads to use and the number of stars involved in the simulation.

We add the following pragma around the second subloop shown in Listing (8):

```
#pragma omp parallel for default(shared) private(jstar)
```

Now, we suggest an alternative parallelism strategy which incorporates OpenMPI. This strategy replaces the existing OpenMP parallelism on the outer loop with an OpenMPI alternative as detailed in the diagram below:

The first strategy (left-hand side) incurs significant overhead due to invoking a parallel region at every timestep, whereas MPI only creates tasks once at the start and maintains them until termination. However, MPI requires

Figure 6: The contour denoted as $\Gamma(T)$

minimising communication calls within the timestep loop to reduce overhead. Despite this, both strategies achieve the same overall parallelism.

3.2 OpenMP Scalability Issue

We tried to use `schedule(dynamic)` over the default `schedule(static)` and this caused timings to become drastically worse so we use a static schedule. With this in place, we can now time our code for different star sizes and threads. Below we give a table of various `nStars` and the optimal number of threads which achieve fastest computation times:

Number of Stars	Optimal Thread Count	Timing
20	2	10.56
50	4	7.77
100	8	14.57
500	28	65.95
1000	28	(Exceeded 120 second time limit)

Table 5: Timing Pure OpenMP implementation for various `nStars` with fixed `Nsteps` = 1000000

Observe that using a pure OpenMP solution isn't easily scalable, at 1000 stars, we exceed the 2 minute time limit barrier with 28 threads, but with 28 cores we could achieve much faster times as seen later. Therefore, using MPI will allow us to produce a much more scalable solution.

3.3 Comparing OpenMP Strategies

We attempt to try parallelising different regions of the code in an attempt to balance between overhead and performance boost. From Figure (4), we predict that parallelising the acceleration loop will provide the best performance increase since it is the most computationally demanding. The following table describes which parts of the serial code we add OpenMP parallelism to and describe the optimal threads that provide the best time. We fix our tests to `nStars` = 50 and `Nsteps` = 1000000 for comparable results.

3.4 OpenMP Performance Measurements & Analysis

The *speedup* ψ , is defined as¹

$$\psi(P) := \frac{\tau(1)}{\tau(P)},$$

¹See <https://www.sdsc.edu/~majumdar/thesis/node47.html> for more information

Strategy	Optimal Thread Count	Timing
Parallelise outer loop of acceleration section only	4	7.77
Parallelise using reduction inner loop of acceleration section only	4	(Exceeded 120 second time limit)
Parallelise both loops described in Listings (7), (8).	4	12.83

Table 6: Timing Pure OpenMP implementation for various strategies

where τ is a function which measures the time it takes the code to run with P threads or processors. The *Karp-Flatt metric* is defined as ²

$$F(P) := \frac{1/\psi(P) - 1/P}{1 - 1/P},$$

where ψ is the speed-up function as defined before.

The *parallel efficiency* ε , is defined as ³

$$\varepsilon(P) := \frac{\psi(P)}{P}.$$

3.4.1 Statistics for 500 Stars on 100000 steps

Threads P	Timing (seconds)	Speed Up ψ	Karp-Flatt Metric F	Efficiency ε
1	87.155853	-	-	-
2	43.803020	1.989722	0.005166	0.994861
4	22.658863	3.846435	0.013308	0.961609
8	11.596888	7.515452	0.009211	0.939431
16	6.509286	13.389464	0.012998	0.836842
24	5.080504	17.154962	0.017348	0.714790
32	5.652596	15.418730	0.034690	0.481835
48	6.326126	13.787734	0.052795	0.287244

Table 7: Timing Pure OpenMP implementation for `nStars = 500` with fixed `Nsteps = 100000`

We plot the above table using Python and display the results below:

- (Timings Plot):** Execution time decreases as the number of threads increases initially but then fluctuates for higher thread counts and eventually starts increasing for large thread counts. This suggests that parallelism is initially effective in reducing computation time, but as the number of threads increases, additional overheads from thread management seem to negate the benefits of parallel execution.
- (Speed-up Plot):** The speed-up initially increases with the number of threads, peaking at $P = 16$ with $\psi(16) = 2.83$. However, at higher thread counts the speed-up decreases. Ideally, we expect speed-up to increase linearly with number of threads in a perfectly parallelisable workload. The observed sub-linear scaling indicates the presence of bottlenecks and we observe that these are overheads

²See https://www.uio.no/studier/emner/matnat/ifi/INF3380/v12/undervisningsmateriale/correct_Karp_Flatt.pdf for more information

³See <https://stackoverflow.com/questions/16262246/calculating-parallel-efficiency> for more information

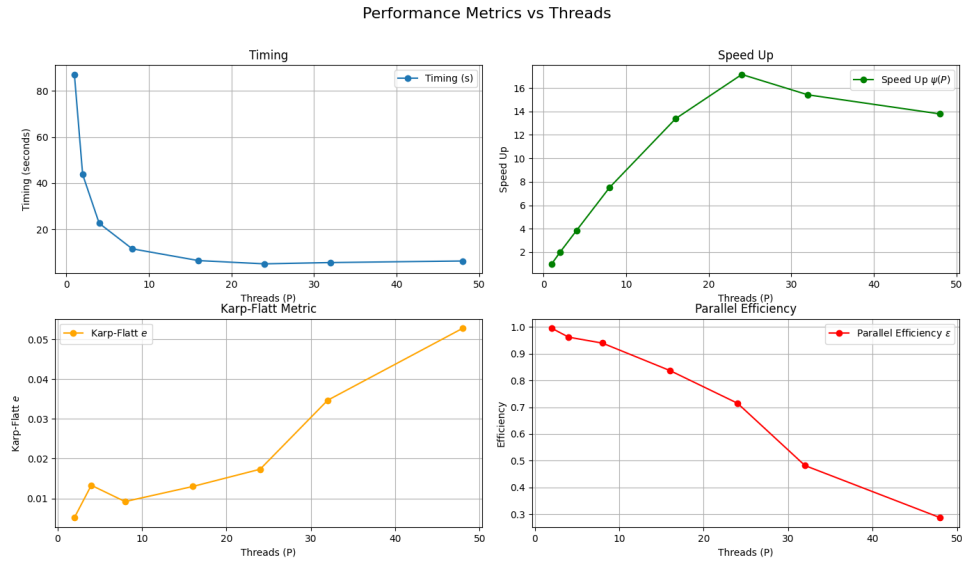


Figure 7: Plots of the timing, speed up ψ (Top left, right), Karp-Flatt metric F and Efficiency ϵ (Bottom left, right).

caused by invoking a large number of parallel regions.

3. **(Karp-Flatt Metric):** The Karp-Flatt metric aims to quantify the amount of workload that is inherently serial. For smaller thread counts, e remains small except at the anomalous cases of specific small threads suggesting that most workload is benefiting from parallelisation. At higher thread counts, e increases significantly suggesting that the parallel overheads are dominating the performance losses.
4. **(Parallel-Efficiency):** Parallel efficiency decreases steadily as the number of threads increases, this suggests the overhead of additional threads outweighs the computational benefits of parallelising. This suggests we gain most benefit from parallelisation at a smaller amount of threads.

3.5 MPI & OpenMP Hybrid Implementation

Our strategy is to split the workload of computing the velocity-Verlet algorithm and acceleration calculations, across a specified number of MPI tasks and add OpenMP parallelism to each task in an attempt to create a hybrid parallelism solution. We outline a general guide for implementing our strategy into the serial version of the code.

Step 1: Include the MPI header file using `# include <mpi.h>` and initialise MPI

```
int rank, size;
MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);
```

Step 2: Verify all arguments have been successfully read on each MPI task. For this, we include some additional code and make use of an `MPI_Allreduce` to see if any contributions of error were picked up from any task. If any task fails to read the data, we terminate the program.

```
int ra = read_args(argc,argv,&nStars,&nSteps,&delta_t,&KE_init,&seedtime,
                  &ClusterMass, &ClusterRadius,&ClusterApogee,&ClusterPerigee);

int all_ra;
MPI_Allreduce(&ra, &all_ra, 1, MPI_INT, MPI_SUM, MPI_COMM_WORLD);
if (all_ra == 0) {
    if (rank == 0) {
        <Success>
    }
} else {
    if (rank == 0) {
        <Failure>
    }
    return EXIT_FAILURE;
}
```

Step 3: We need to verify the number of processors is less than or equal to `nStars` so that no processor works on 0 stars which could cause problems when using `Scatterv` and `Gatherv` later on.

```
if (size > nStars) {
    if (rank == 0) {
        printf("Error: Number of processors must be less than number of stars\n");
    }
    return EXIT_FAILURE;
}
```

Step 4: We allocate memory for both global arrays such as `mass`, `radius`, `pos`, `vel`, `acc` and also for local buffers such as `localMass`, `localRadius`, `localAcc`, `localPos`, `localVel`, `localActive`. The code for this is basic so we omit it here.

Step 5: We setup `displacements` and `sendCounts` arrays which distribute the workload to each processor accounting for remainders too so that we can use `MPI_Scatterv` and `MPI_Gatherv` later on.

```
// Calculate number of stars per processor
int chunkSize = nStars / size;
int remainder = nStars % size;
int localNStars = chunkSize + (rank < remainder ? 1 : 0);

int* displacements = malloc(size * sizeof(int));
int* displacements3 = malloc(size * sizeof(int));
int* sendCounts = malloc(size * sizeof(int));
int* sendCounts3 = malloc(size * sizeof(int));
```

```

if (displacements==NULL) {printf("Error allocating displacements array") ; exit(EXIT_FAILURE);}
if (displacements3==NULL) {printf("Error allocating displacements3 array") ; exit(EXIT_FAILURE);}
if (sendCounts==NULL) {printf("Error allocating sendCounts array") ; exit(EXIT_FAILURE);}
if (sendCounts3==NULL) {printf("Error allocating sendCounts3 array") ; exit(EXIT_FAILURE);}

int offset = 0;
int offset3 = 0;

for (int i = 0; i < size; i++) {
    sendCounts[i] = chunkSize + (i < remainder ? 1 : 0);
    sendCounts3[i] = sendCounts[i] * 3;

    displacements[i] = offset;
    displacements3[i] = offset3;

    offset += sendCounts[i];
    offset3 += sendCounts3[i];
}

```

Step 6: We initialise the star data on rank 0 only.

```

if (rank == 0) {
    initialiseCluster(nStars,KE_init,ClusterMass,BlackHoleMass,
                    ClusterRadius,ClusterApogee,ClusterPerigee,
                    mass,radius,pos,vel,acc);
}

```

Step 7: We output diagnostics on rank 0 only by encapsulating the appropriate serial code with an if statement.

Step 8: We scatter the data of stars to each processor.

```

/*-----*/
/* Distribute data across processors */
/*-----*/
MPI_Scatterv(mass, sendCounts, displacements, MPI_DOUBLE, localMass, localNStars, MPI_DOUBLE, 0,
            MPI_COMM_WORLD);
MPI_Scatterv(radius, sendCounts, displacements, MPI_DOUBLE, localRadius, localNStars, MPI_DOUBLE, 0,
            MPI_COMM_WORLD);
MPI_Scatterv(acc, sendCounts3, displacements3, MPI_DOUBLE, localAcc, localNStars * 3, MPI_DOUBLE, 0,
            MPI_COMM_WORLD);
MPI_Scatterv(pos, sendCounts3, displacements3, MPI_DOUBLE, localPos, localNStars * 3, MPI_DOUBLE, 0,
            MPI_COMM_WORLD);
MPI_Scatterv(vel, sendCounts3, displacements3, MPI_DOUBLE, localVel, localNStars * 3, MPI_DOUBLE, 0,
            MPI_COMM_WORLD);

// Broadcast mass data to all other processors, used inside inner loop for acceleration
MPI_Bcast(mass, nStars, MPI_DOUBLE, 0, MPI_COMM_WORLD);

```

Step 9: We make changes to the for loops so they only iterate over the number of stars that have been assigned to that processor and the next key step is invoking an `MPI_Allgatherv` to collect positional data from all processors and broadcast it to each processor so that we can carry out the inner loop of acceleration which requires the updated positions of all stars.

```

// Gatherv and then Bcast positional data to all processors, used for inner loop for acceleration
MPI_Allgatherv(localPos, localNStars * 3, MPI_DOUBLE, pos, sendCounts3, displacements3, MPI_DOUBLE,
              MPI_COMM_WORLD);

```

Step 10: We move OpenMP parallelism from the outer loop on acceleration to the inner loop since the outer loop has

already had its workload split across the MPI tasks and thus it makes sense to apply the OpenMP parallelism to split the workload across the inner loop which still deals with all the stars.

```
#pragma omp parallel for shared(pos, mass, mypos, nStars) private(jstar) reduction(+ : xsum, ysum,
    zsum) schedule(static)
for (jstar = 0; jstar < nStars; jstar++) {
    ...
}
```

Step 11: We modify the book keeping loop so that it uses local buffers for active and mass of the stars the processor is working on. The key step here is that we use `MPI_Reduce` to determine the minimum distance and the total number of active stars across all processors.

```
/*-----*/
/ Bookkeeping /
/*-----*/
for (istar=0; istar<localNStars; istar++) {
    ...
}
// Sum number of active stars across processors and get min distance
MPI_Reduce(&localNactive, &totalNactive, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
MPI_Reduce(&minRadsq, &minRadsqGlobal, 1, MPI_DOUBLE, MPI_MIN, 0, MPI_COMM_WORLD);
```

Step 12: After the timesteps loop is complete for each processor, we can optionally gather all the acceleration and velocity data from each processor to rank 0, although this isn't required since we no longer require this data.

```
// Gather acceleration and velocity from all processors, not needed
MPI_Gatherv(localVel, localNStars * 3, MPI_DOUBLE, vel, sendCounts3, displacements3, MPI_DOUBLE, 0,
    MPI_COMM_WORLD);
MPI_Gatherv(localAcc, localNStars * 3, MPI_DOUBLE, acc, sendCounts3, displacements3, MPI_DOUBLE, 0,
    MPI_COMM_WORLD);
```

Step 13: We free up all the memory allocations we used and finally call the `MPI_Finalize()` command on each rank and this completes our hybrid implementation.

3.6 MPI & OpenMP Hybrid Performance

The specification states we should try to test our solution for $nStars = 500, 600$ and 750 with $Nsteps = 100000$.

3.6.1 Statistics for 500 Stars on 100000 steps

Number of Processors P	Simulation Time (s)	Speed up ψ	Karp-Flatt Metric F	Efficiency ε
1	106.334662	-	-	-
2	53.562472	1.985246	0.007432	0.992623
4	27.268279	3.899574	0.008584	0.973935
8	14.394908	7.386964	0.011856	0.923371
16	7.561930	14.061842	0.009189	0.878865
32	4.577103	23.231870	0.012175	0.725996
48	4.229581	25.140708	0.019346	0.523765
64	4.694239	22.652162	0.028974	0.353940
96	5.025135	21.160558	0.037229	0.220422

Table 8: MPI-OpenMP Hybrid with multi-threading disabled

We also experiment with multiple OpenMP threads with a set number of cores to see if using OpenMP threads in conjunction with MPI tasks provides better speedups.

Number of Processors P	Number of Threads per processor T	Simulation Time (s)	Speed up ψ	Karp-Flatt Metric F	Efficiency ε
24	4	6.331728	16.793940	0.049646	0.174937
32	3	5.129470	20.730146	0.038220	0.215939
48	2	4.191278	25.370463	0.029305	0.264276

Table 9: MPI-OpenMP Hybrid with multi-threading enabled, **96** total cores used

Number of Processors P	Number of Threads per processor T	Simulation Time (s)	Speed up ψ	Karp-Flatt Metric F	Efficiency ε
16	4	4.216435	25.219092	0.024409	0.394048
32	2	4.873435	21.819243	0.030686	0.340926

Table 10: MPI-OpenMP Hybrid with multi-threading enabled, **64** total cores used

Number of Processors P	Number of Threads per processor T	Simulation Time (s)	Speed up ψ	Karp-Flatt Metric F	Efficiency ε
16	3	7.098426	14.980034	0.046899	0.312084
24	2	6.903272	15.403516	0.045025	0.320907

Table 11: MPI-OpenMP Hybrid with multi-threading enabled, **48** total cores used

3.6.2 Statistics for 600 Stars on 100000 steps

Number of Processors P	Simulation Time (s)	Speed up ψ	Karp-Flatt Metric F	Efficiency ε
1	174.230015	-	-	-
2	89.448108	1.947833	0.026782	0.973917
4	44.299779	3.932977	0.005680	0.983244
8	22.583290	7.714997	0.005277	0.964375
16	12.055476	14.452355	0.007139	0.903272
32	7.193701	24.219802	0.010362	0.756869
48	6.596391	26.412930	0.0173892	0.550269
64	6.590260	26.437502	0.022552	0.413086
96	4.748328	36.692919	0.017014	0.382218

Table 12: MPI-OpenMP Hybrid with multi-threading disabled

Number of Processors P	Number of Threads per processor T	Simulation Time (s)	Speed up ψ	Karp-Flatt Metric F	Efficiency ε
24	4	8.412910	20.709839	0.038268	0.215727
32	3	6.388864	27.270809	0.026529	0.284071
48	2	5.144568	33.866792	0.019312	0.352779

Table 13: MPI-OpenMP Hybrid with multi-threading enabled, **96** total cores used

Number of Processors P	Number of Threads per processor T	Simulation Time (s)	Speed up ψ	Karp-Flatt Metric F	Efficiency ε
16	4	9.978980	17.459702	0.042311	0.272808
32	2	7.619583	22.866083	0.028554	0.357283

Table 14: MPI-OpenMP Hybrid with multi-threading enabled, **64** total cores used

Number of Processors P	Number of Threads per processor T	Simulation Time (s)	Speed up ψ	Karp-Flatt Metric F	Efficiency ε
16	3	8.894269	19.589020	0.030859	0.408105
24	2	6.001186	29.032597	0.013900	0.604846

Table 15: MPI-OpenMP Hybrid with multi-threading enabled, **48** total cores used

3.6.3 Statistics for 750 Stars on 100000 steps

Number of Processors P	Simulation Time (s)	Speed up ψ	Karp-Flatt Metric F	Efficiency ε
1	264.593564	-	-	-
2	133.806293	1.977437	0.011410	0.988719
4	68.392645	3.868743	0.011309	0.967186
8	34.272902	7.720197	0.005178	0.965025
16	17.655523	14.986447	0.004509	0.936653
32	9.977699	26.518495	0.006668	0.828703
48	9.177335	28.831198	0.014146	0.600650
64	8.103235	32.652831	0.015238	0.510200
96	5.603083	47.222853	0.010873	0.491905

Table 16: MPI-OpenMP Hybrid with multi-threading disabled

Number of Processors P	Number of Threads per processor T	Simulation Time (s)	Speed up ψ	Karp-Flatt Metric F	Efficiency ε
24	4	10.023768	26.396617	0.027756	0.274965
32	3	8.410918	31.458345	0.021596	0.327691
48	2	6.799461	38.913903	0.015442	0.405353

Table 17: MPI-OpenMP Hybrid with multi-threading enabled, **96** total cores used

Number of Processors P	Number of Threads per processor T	Simulation Time (s)	Speed up ψ	Karp-Flatt Metric F	Efficiency ε
16	4	13.466833	19.647794	0.035831	0.306997
32	2	8.080110	32.746282	0.015150	0.511661

Table 18: MPI-OpenMP Hybrid with multi-threading enabled, **64** total cores used

Number of Processors P	Number of Threads per processor T	Simulation Time (s)	Speed up ψ	Karp-Flatt Metric F	Efficiency ε
16	3	11.425384	23.158396	0.022823	0.482467
24	2	8.444317	31.333921	0.011317	0.652790

Table 19: MPI-OpenMP Hybrid with multi-threading enabled, **48** total cores used

3.6.4 Plots of speed up, Karp-Flatt Metric and efficiency against processors

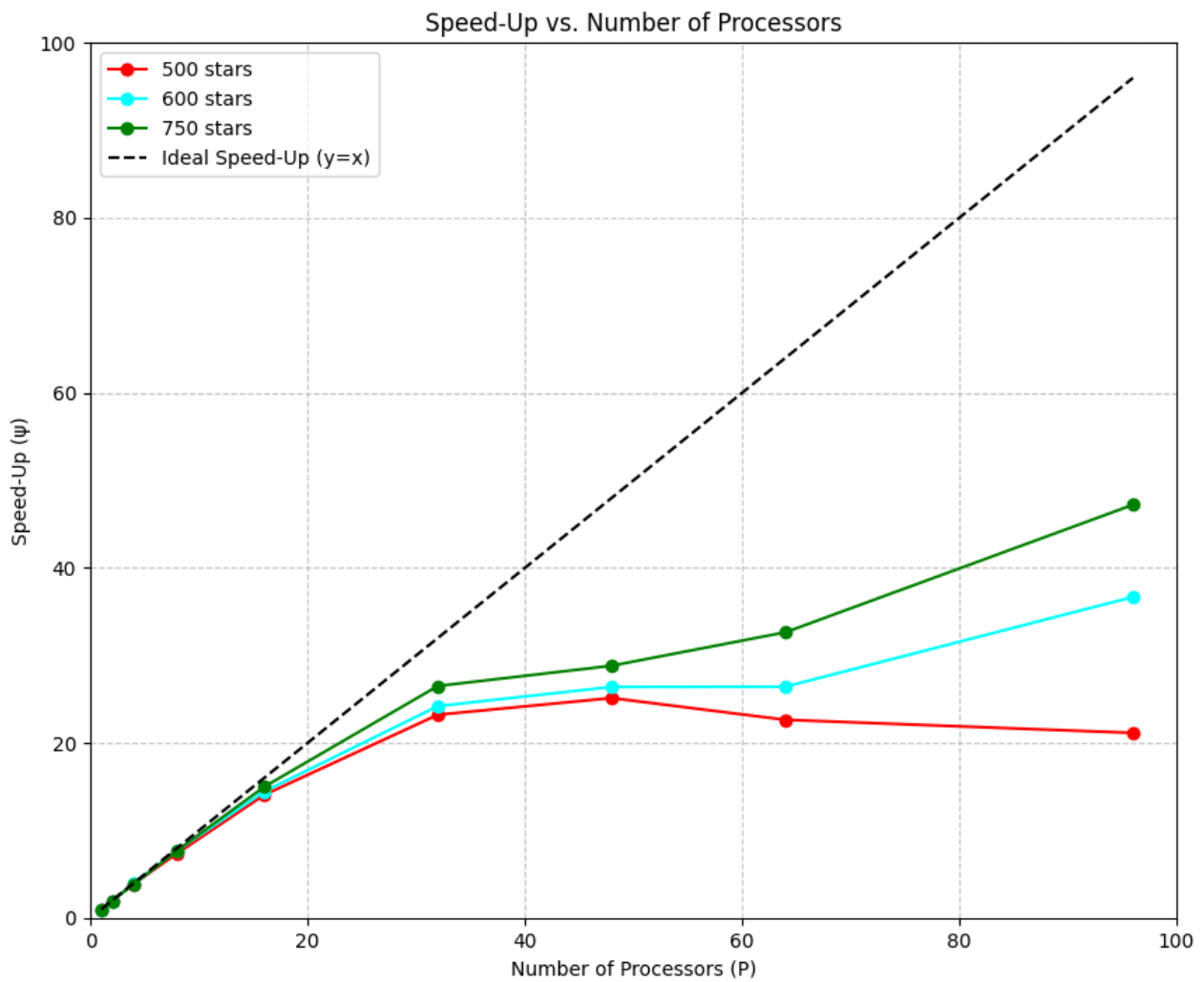


Figure 8: How speed up varies with the number of processors used for 500, 600 and 750 stars

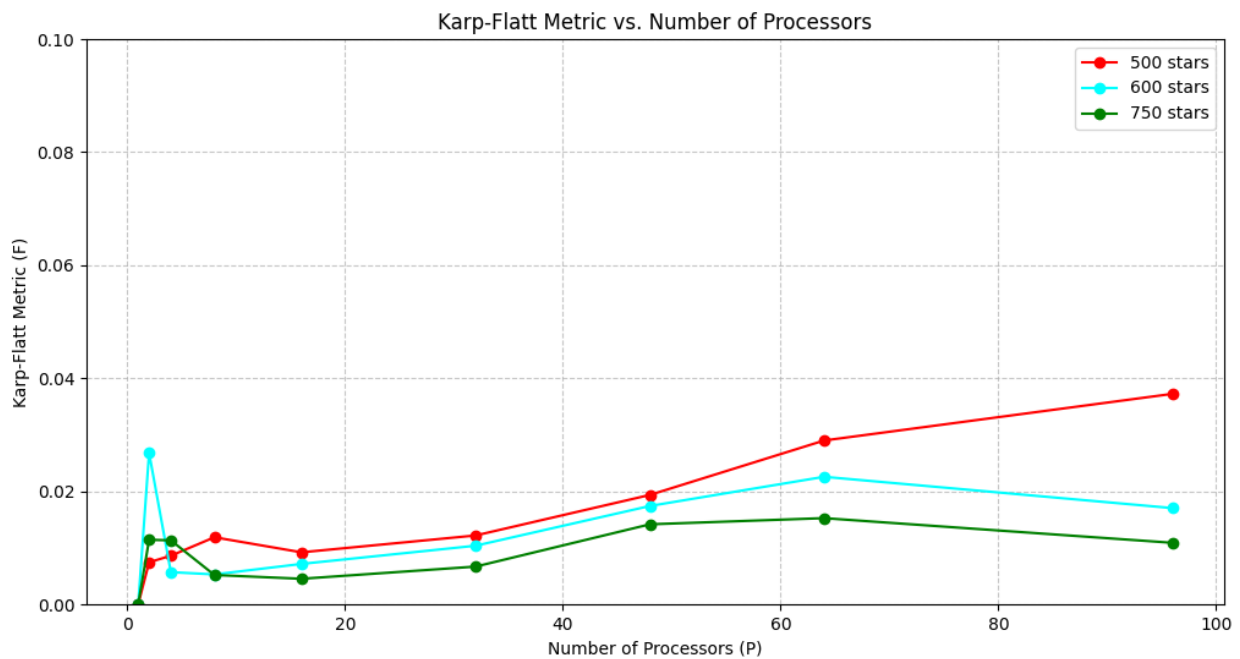


Figure 9: How Karp-Flatt Metric varies with the number of processors used for 500, 600 and 750 stars

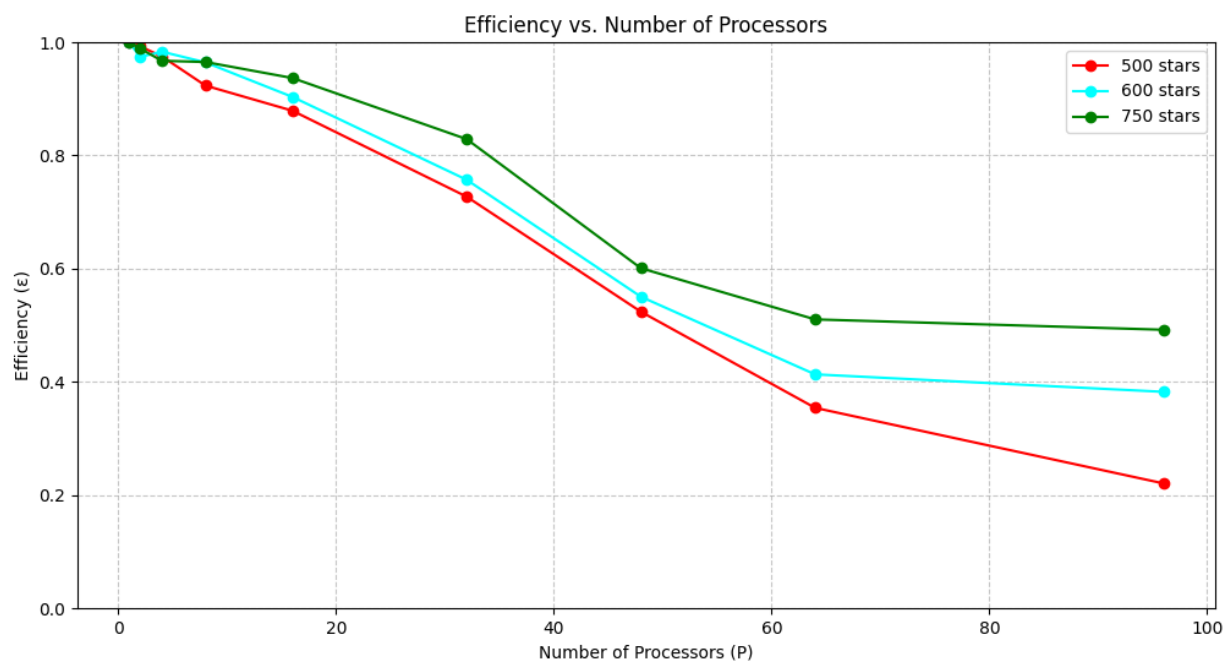


Figure 10: How parallel efficiency varies with the number of processors used for 500, 600 and 750 stars

3.6.5 Analysis of plots

The Karp-Flatt metric F helps provide insights into the limitations of parallel scaling. It measures the fraction of execution time spent in either serial regions of the code or in parallel overheads such as inter-processor communications through MPI or when invoking parallel regions using OpenMP. In our code, we parallelised every loop including the book keeping section. Therefore, in our code the Karp-Flatt metric measures the fraction of time lost to parallel overheads and inter-processor communications only.

For our 500 star graph, F increases with the number of processors reaching a 0.037 peak at 96 processors. The increase suggests that parallel overheads become a limiting factor for us as processor counts increase. This is supported by the speed-up graph where after 32 processors our speed-up starts to get limited. Furthermore, the parallel efficiency graph also suggests for large processor counts the efficiency decreases drastically which suggests overheads.

For 600 and 750 stars we observe a similar pattern but the limiting factoring happens at a higher processor count, this suggests that as the problem size increases, the impact of parallel overheads diminishes.

3.7 Full Scale Simulation

Number of threads P	1	1	1	3	2	1
Number of processors P	24	48	64	32	48	96
Simulation Time (s)	38.343356	23.698404	22.705864	19.203940	16.979058	16.417479

Table 20: MPI-OpenMP Hybrid full scale simulation on 1500 stars

We observe that using a full 96 MPI processors achieves the best time for processing 1500 stars. However, from our previous timings, we observe the difference in using 48 processors and 2 threads and just 96 processors, decreases as the problem size increases (we discussed this relationship earlier). Therefore, we can expect our hybrid solution to be more efficient using 48 processors and 2 threads when we increase the problem size to beyond 1500 stars.

We fix the simulation time to be 1000000 years and vary the timesteps to be $Y * 0.00001$, $Y * 0.0001$ and $Y * 0.000001$ where Y denotes **SecondsIn1Myear**. The following plot displays the behaviour of the distance from the nearest star to the black hole over time. We use the following parameters for our simulation: `./gravity -N 100 -M 100 -A 10 -P 2 -R 0.2`.



Figure 11: Distance of closest star to black hole across 1000000 years for various timesteps.

Using the default parameters: `./gravity -N 100` over 1000000 years and varying the timesteps in the same way, we yield the following graph:

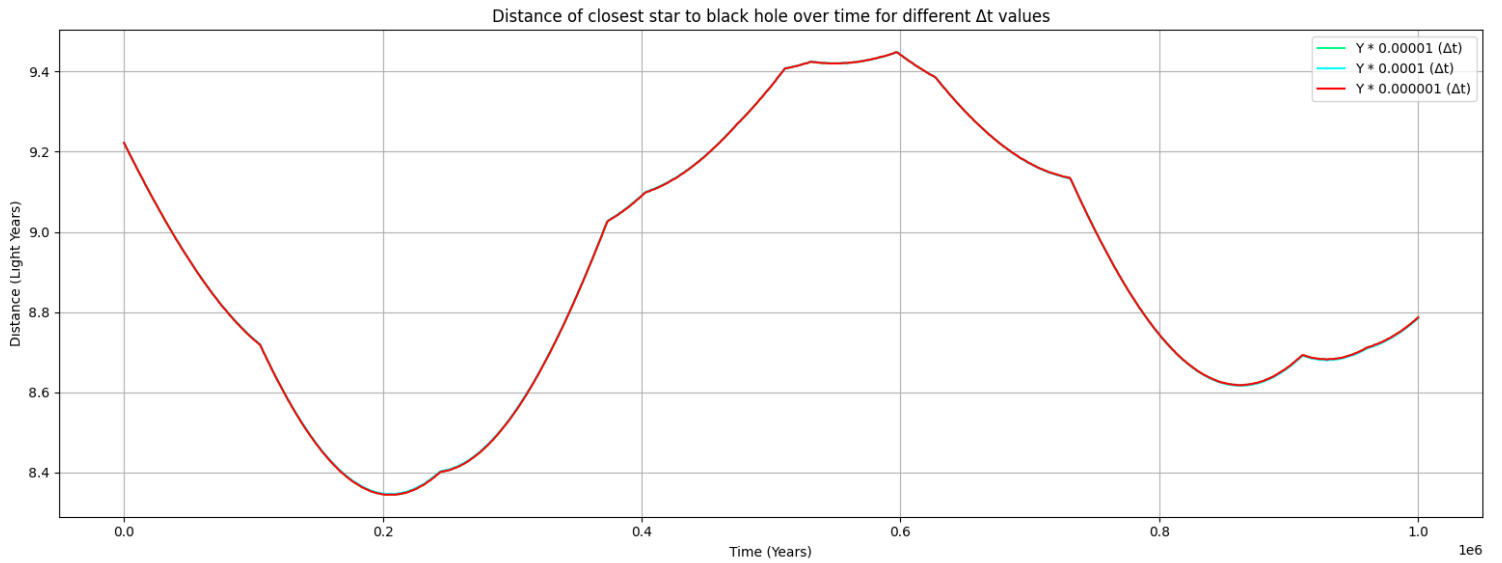


Figure 12: Distance of closest star to black hole across 1000000 years for various timesteps.

This suggests that the default parameters yield a stable simulation where as the parameters used before provide an unstable simulation since changing the timestep slightly causes dramatic changes in the output of the simulation.

We can also use these above graphs to verify the correctness of our code. We know our OpenMP implementation is correct since we add one pragma to the correct serial implementation. Therefore, we can run the above simulations on our pure OpenMP implementation and see if we yield the same output graphs. Indeed doing so yields the same graphs as above (which were produced using our hybrid MPI-OpenMP code) which implies the correctness of our code.

4 Conclusion

In this conclusive section, we aim to report on the key parts of our simulation.

Detail the parallelism strategy we have chosen: Our parallelism strategy employs a hybrid approach combining MPI and OpenMP. We use MPI to distribute the workload and handle communication of star data between processes, while OpenMP is applied to parallelise within each process. Since the outer loop of the acceleration section is distributed across tasks, we focus OpenMP parallelism on the inner loop of the acceleration section. Additionally, we avoid parallelising every loop to minimise overhead and maintain performance. To further optimise, we limit MPI communication calls within the timesteps loop to reduce intercommunication overhead.

Why is this parallelism strategy appropriate and efficient: As previously discussed, this strategy minimises parallel overhead and reduces inter-communication calls within the timesteps loop while optimizing the performance of our code. Given that most of the code is independent, except for the acceleration section, adopting an MPI-based approach aligns well with our strategy.

Describe any problems we had implementing the simulation in parallel: Our parallel implementation encountered issues with compute time for small numbers of stars, where the serial code outperformed it due to the parallel overhead dominating the computational workload. However, as the number of stars increases, the computations grow quadratically and eventually surpass the parallel overhead, indicating that using more cores improves parallel efficiency. This is reflected in our performance measurements, which show peak performance with 500, 600, and 750 stars occurring around 32 processors, followed by a rapid decline in efficiency. The trend indicates that parallel efficiency improves as the number of stars increases, supporting our hypothesis.

Discuss the stability of different types of simulation: From the two graphs above, depicting the distance of the closest star to the black hole, we deduce that one simulation is more stable than the other. The simulation with default parameters demonstrates stability even when the timestep is adjusted, whereas the custom example exhibits instability. These represent two distinct simulations, each displaying different levels of stability.

Would it be feasible to have an adaptive timestep depending on the current system parameters: An adaptive timestep based on the systems parameters is certainly feasible. A smaller timestep offers greater accuracy, particularly when simulating a system that evolves rapidly over time. However, it is also more computationally expensive, whereas a larger timestep allows for a more efficient simulation. Striking a balance between these two is crucial. By evaluating the systems parameters, we can identify when the system is likely to evolve rapidly and adjust the timestep accordingly—using a smaller timestep in such cases. Conversely, for a slower-evolving system, a larger timestep can be employed to reduce computational costs. Given that our setup has the potential to change dynamics based on its parameters, implementing an adaptive timestep is both practical and viable.

Discuss the scaling of the code with respect to the number of stars and the number of timesteps: The scaling of the code depends significantly on the number of stars and number of timesteps:

1. **Scaling with the number of stars:** As the number of stars N increases, the computational workload grows quadratically due to the $\mathcal{O}(N^2)$ complexity of the pairwise interactions in the acceleration calculation. This scaling benefits parallelism since the larger workload offsets the overhead introduced by communication and thread management. For a small number of stars, the parallel overhead dominates, leading to slower performance compared to the serial implementation. However, as the number of stars increases, the computations outweigh the overhead, resulting in better parallel efficiency and scalability. This makes scaling the code easier.
2. **Scaling with the Number of timesteps:** The number of timesteps impacts the overall runtime linearly since each timestep involves a fixed set of computations for a given number of stars. Unlike an increased number of stars, increasing the number of timesteps also increases the parallel overhead since the OpenMP regions are invoked more times. This makes scaling the code harder.